



Ultimate

LANGGRAPH

CHEATSHEET



Your Pocket Guide for LangGraph

► Presented By:



Shubham Kansal
Founder | Mowka

Basic Setup

```
pip install langchain-openai

from langchain_openai import ChatOpenAI
model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7
)
```

Simple Chat Completion

```
from langchain_openai import ChatOpenAI

chat = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7
)

response = chat.invoke("What is LangChain?")
print(response.content)
```

PROMPT TEMPLATE

BASIC PROMPT TEMPLATE

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Write a poem about {topic}"
)

print(prompt.format(topic="Python"))
```

MULTIPLE INPUT VARIABLES

```
prompt = PromptTemplate.from_template(
    f"Explain {concept} in simple terms with an example about {example_topic}."
)

print(prompt.format(
    concept="Functions",
    example_topic="cooking"
))
```



PROMPT TEMPLATE

CHATPROMPTTEMPLATE

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert developer."),
    ("human", "Explain {topic} with a simple example.")
])

formatted = prompt.format_messages(topic="APIs")
print(formatted)
```

STRUCTURED OUTPUT PROMPTS

```
prompt = PromptTemplate.from_template(
    """
    Provide output in JSON only.
    Name: {name}
    Age: {age}
    Return fields: summary, category
    """
)

print(prompt.format(name="Alice", age=28))
```



Chains

BASIC LLMCHAIN

```
from langchain.chains import LLMChain
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template("Tell me a joke about {topic}")
chain = LLMChain(llm=llm, prompt=prompt)

chain.run("Python")
```

CHAIN WITH TWO INPUTS

```
prompt = PromptTemplate.from_template(
    "Explain {concept} using a {example_type} example."
)
chain = LLMChain(llm=llm, prompt=prompt)

chain.run({"concept": "loops", "example_type": "cooking"})
```



LangChain LCEL

BASIC LCEL CHAIN

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

chain = (
    PromptTemplate.from_template("Tell me a joke about {topic}.")
    | llm
    | StrOutputParser()
)

chain.invoke({"topic": "Python"})
```

TWO-STEP SEQUENTIAL PIPELINE

```
translate = (
    PromptTemplate.from_template("Translate to Hindi:\n{text}")
    | llm
    | StrOutputParser()
)

summarize = (
    PromptTemplate.from_template("Summarize in one line:\n{text}")
    | llm
    | StrOutputParser()
)

flow = translate | summarize

flow.invoke({"text": "LangChain helps developers build AI applications."})
```

Agents and Tools

```
from langchain.tools import Tool
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

# Simple tool
def calc(x):
    return str(eval(x))

calculator = Tool(
    name="Calculator",
    func=calc,
    description="Do basic math."
)

agent = initialize_agent(
    tools=[calculator],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION
)

agent.invoke({"input": "What is 5 * 12?"})
```



Memory

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

conversation = ConversationChain(
    llm=llm,
    verbose=True,
    memory=ConversationBufferMemory()
)

conversation.predict(input="How can one overcome anxiety?")
```

Document Loading

```
from langchain_community.document_loaders import TextLoader, PyPDFLoader

docs = TextLoader("file.txt").load()
pdf_docs = PyPDFLoader("file.pdf").load()
```



Embeddings + Vector Store

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vector_store = FAISS.from_documents(chunks, embeddings)

results = vector_store.similarity_search("Explain LangChain", k=3)
```

Retrieval-based Question Answering

```
from langchain.chains import RetrievalQA

retriever = vector_store.as_retriever()

qa = RetrievalQA.from_chain_type(
    llm=model,
    retriever=retriever,
    chain_type="stuff"
)

print(qa.invoke({"query": "What does the PDF say about AI?"})["result"])
```



Shubham Kansal

Founder | Mowka

 mowka.in



Follow